

Denormalization and NoSQL Modelling P2

Part 1 Denormalization:

The purpose of this task is to identify possible denormalization opportunities in the provided schema and explaining when and why denormalization would be appropriate for use.

Well, the database is very organized and split into many tables such as user, advertiserClient, contract, interest, ect. This is called normalization which helps avoid repeating data and keeps everything nice and clean.

However, in this cause because the data is split into many different tables, we first need to join several tables together when we want to get information, however too many joins can slow things down especially if the system reads data often.

I think one possible solution would be to combine the user and zip tables. If we often need to show a user's city and state, we could store that info directly inside the user table instead of joining the zip table every time

Another example of this is advertiserClient and contact, if contact info is always used together with advertiser details , we could store the contact info directly inside the advertiserClient table which would reduce the number of joins.

Same would apply to user interest since getting a user's interest might require joining multiple tables, by doing this we could store some of that info directly in the user table to make the queries faster

Overall, I think denormalization is useful when the system read data a lot and performance is important, however it increase repeated data and can cause problems if info needs to be updated in many places.

Part 2 (Bike Store NoSQL Modelling) :

Task 1 Aggregate Boundary :

Based on this ERD , the aggregate boundary is defined around the order entity. This order represents a complete business transaction and connects multiple related entities including customer, staff, store, order_items and products

Based of this in the relational model, retrieving full order details requires joining several tables such as order order_items, products, costumers, stores and staffs. Since this data is typically accessed together when viewing an order, it makes sense to group it within a single aggregate in an SQL model

And so, the order entity is chosen as the aggregate root which includes all the related data, customer info, store details, staff details, and order items with product info

Overall, this design reduces the need for join and follows database principles where related data that is frequently retrieved together should be store together

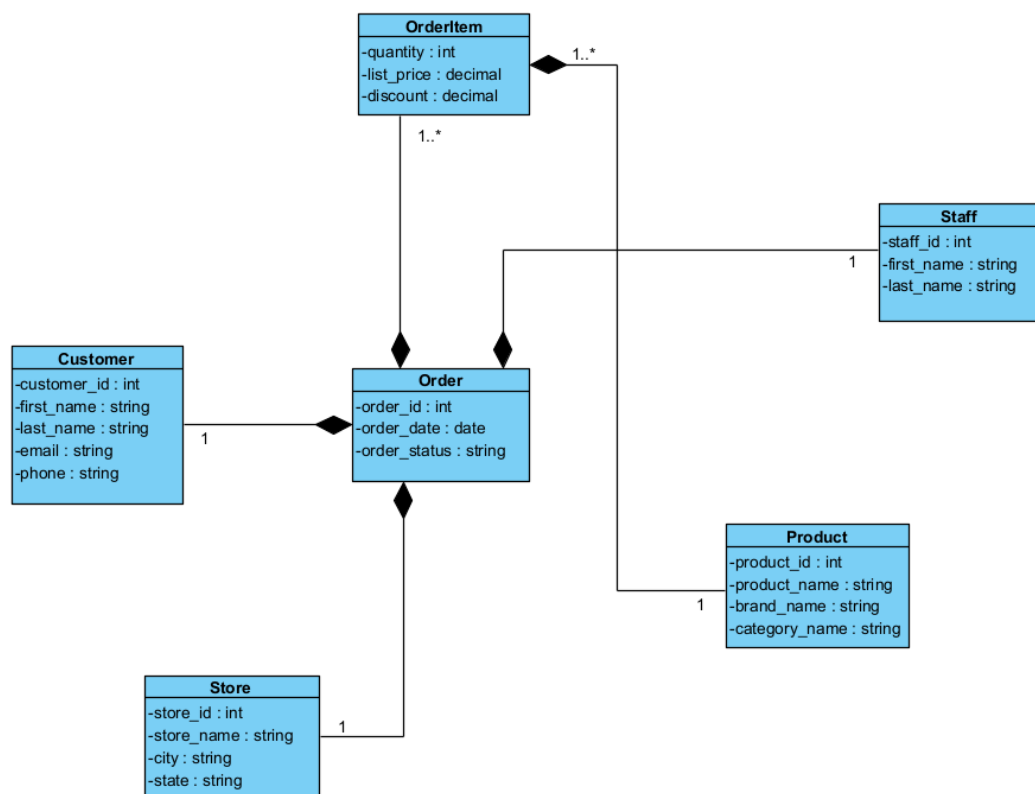
Task 2 Aggregate data model :

With the knowledge of the previous task knowing Order being the root of the diagram, I created the ULM diagram which represents the structure of the order aggregate

The order class which we know is the root of the aggregate contains, order_id, order_date and order_status. The order also has a composition relationship with customer(1 per order), store (1 per order), staff and orderItem(one or more per order)

Each orderItem contains quantity, list_price and discount, each orderItem also contains one product which includes product_id, product_name, brand_name and category_name

This use of composition in the diagram shows these objects exist only within the Order document



Task 3 JSON Files:

This next JSON file represents one Order aggregate based on the UML diagram I designed, each line of text contains one complete order object with embedded customer, store, staff and order item info.

This overall structure shows how the data is stored in a database.

```
1  {
2      "order_id": 1001,
3      "order_date": "2024-05-15",
4      "order_status": "complete",
5
6      "customer": {
7          "customer_id": 201,
8          "first_name": "John",
9          "last_name": "Smith",
10         "email": "John.Smith@gmail.com",
11         "phone": "0123456789"
12     },
13
14     "store": {
15         "store_id": 1,
16         "store_name": "Central bike store",
17         "city": "London",
18         "state": "Greater London"
19     },
20
```

```
    "staff" : {
        "staff_id": 10,
        "first_name": "Sarah",
        "last_name": "Brown"
    },
    "order_items": [
        {
            "quantity": 1,
            "list_price": 1200.00,
            "discount": 0.10,
            "product": {
                "product_id": 301,
                "product_name": "Mountain Bike x100",
                "brand_name": "trailMaster",
                "category_name": "mountain bikes"
            }
        }
    ],
}
```

```
40     {
41         "quantity": 2,
42         "list_price": 45.00,
43         "discount": 0.00,
44         "product": {
45             "product_id": 450,
46             "product_name": "Cycling Helmet Pro",
47             "brand_name": "SafeRide",
48             "category_name": "Accessories"
49         }
50     }
51 ]
52 }
53 }
```